

**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
FACULDADE DE ENGENHARIA DE SOFTWARE**

**CAIO DOS SANTOS CUNHA
GUILHERME GIUSEPE PIOVEZAN
JHENIFER LAÍS BARBOSA**

**Inteligência Artificial e Aprendizado de Máquina -
Avaliação Prática 2**

**CAMPINAS
2026**

Sumário

1	INTRODUÇÃO	1
2	FUNDAMENTAÇÃO TEÓRICA	2
2.1	Conceitos de Aprendizado de Máquina	2
2.2	Algoritmos de Classificação Empregados	2
2.3	Métricas de Avaliação de Desempenho	3
3	METODOLOGIA E VALIDAÇÃO	5
3.1	Papel da Validação no Desempenho do Classificador	5
4	ANÁLISE DOS RESULTADOS	7
4.1	Comparação de Otimizadores (SGD e Adam)	7
4.2	Discussão do Impacto da Validação e do Treinamento	8
5	CONCLUSÃO	11
	REFERÊNCIAS	12

1 INTRODUÇÃO

Diferenciar um gato de um cachorro ou um carro de um caminhão é algo natural para nós, mas um grande desafio para os computadores, que enxergam imagens apenas como uma enorme matriz de números (*pixels*). A área da Inteligência Artificial que busca solucionar esse problema é a Visão Computacional.

Este projeto tem como objetivo central construir um sistema automatizado capaz de classificar imagens corretamente por meio de uma Rede Neural Convolucional (CNN). Para isso, utilizamos o *dataset* CIFAR-10, composto por 60 mil imagens digitais divididas em 10 categorias, que incluem desde animais até meios de transporte.

O principal problema de pesquisa deste trabalho é evitar que a máquina apenas “decore” os dados de treino (fenômeno conhecido como *overfitting*), perdendo a capacidade de reconhecer novas imagens no futuro. Nossa hipótese é que a aplicação rigorosa de uma etapa de validação serve como um controle de qualidade essencial para ajustar o modelo e garantir sua capacidade de generalização.

Para alcançar esse resultado, os objetivos específicos compreendem:

- **Construir o pipeline de dados:** realizar o carregamento, a normalização e a ampliação artificial das imagens (*data augmentation*);
- **Desenvolver a arquitetura:** estruturar uma rede convolucional eficiente para extrair características visuais das fotos;
- **Otimizar o treinamento:** monitorar o aprendizado utilizando critérios de parada precoce (*early stopping*) para salvar o melhor classificador;
- **Avaliar o desempenho final:** analisar os erros e acertos no conjunto de teste por meio de métricas estatísticas e da matriz de confusão.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Conceitos de Aprendizado de Máquina

O Aprendizado de Máquina Supervisionado (*Supervised Machine Learning*) funciona de maneira muito semelhante ao processo de ensino de uma criança. Nós apresentamos ao computador um grande conjunto de exemplos (imagens) acompanhados de suas respectivas "respostas corretas" (as etiquetas ou rótulos, como "gato", "cachorro" ou "navio"). A máquina analisa exaustivamente esses dados e tenta encontrar padrões matemáticos que conectem as características visuais da imagem à sua categoria correspondente.

O grande desafio desse processo é no equilíbrio do aprendizado, que pode falhar de duas maneiras principais:

- **Underfitting (Subajuste):** Ocorre quando o modelo é simples demais e não consegue aprender sequer os padrões básicos dos dados. É o equivalente a um aluno que não estudou o suficiente e vai mal tanto nos exercícios de sala de aula quanto na prova final.
- **Overfitting (Sobreajuste):** É o problema central que este projeto combate. Ocorre quando a máquina decora perfeitamente as imagens de treino (inclusive os ruídos e detalhes irrelevantes, como a cor do fundo), mas não aprende a essência matemática do objeto. É o equivalente a um aluno que decorou as perguntas do livro didático: se você mudar uma única palavra ou o número da questão na prova, ele erra, pois não aprendeu a matéria de verdade, apenas memorizou o cenário.

O objetivo do nosso projeto é o oposto: a **Generalização**, que é a capacidade de a inteligência artificial olhar para uma foto completamente inédita (que ela nunca viu antes) e, ainda assim, identificar o objeto corretamente baseando-se no aprendizado real dos formatos e texturas.

2.2 Algoritmos de Classificação Empregados

O classificador desenvolvido e empregado neste experimento foi uma **Rede Neural Convolucional** (CNN, do inglês *Convolutional Neural Network*), batizada em nosso código como `Model`. Redes neurais são modelos matemáticos inspirados na estrutura biológica do cérebro humano, compostos por camadas de neurônios artificiais conectados que processam a informação em estágios.

Diferente de algoritmos tradicionais que exigem que um humano diga quais características procurar em uma imagem, as CNNs possuem a capacidade de descobrir sozinhas o que é mais importante em uma foto através de camadas especializadas:

- **Camadas Convolucionais (*Conv2d*):** Funcionam como lentes ou filtros que deslizam pela imagem procurando elementos visuais específicos. As primeiras camadas

aprendem a detectar traços simples (como linhas retas e bordas); as intermediárias identificam formas geométricas (círculos, curvas); e as finais reconhecem partes complexas do objeto (como rodas de carros ou orelhas de animais).

- **Normalização em Lote (*BatchNorm2d*):** Atua como um "estabilizador" emocional dos dados, reajustando as escalas numéricas dos pixels após cada passagem para que o treinamento ocorra de forma muito mais rápida e matematicamente estável.
- **Agrupamento Máximo (*MaxPool2d*):** É um redutor de tamanho. Ele divide a imagem em pequenas regiões e extrai apenas o pixel de maior valor de cada uma. Isso serve para reduzir o peso computacional e garantir que o modelo identifique a característica não importa em qual canto da imagem ela esteja.
- **Abandono (*Dropout*):** É uma técnica de regularização altamente eficaz. Durante cada rodada de treino, desligamos aleatoriamente uma porcentagem dos neurônios (no nosso caso, 30% e 50% nas camadas finais). Isso força a rede a não depender de um único neurônio "genial" e distribui o aprendizado de forma robusta por todo o sistema, mitigando fortemente o *overfitting*.
- **Camadas Densas / Totalmente Conectadas (*Linear* / *Head*):** Ficam localizadas no final da rede. Elas pegam todas as características visuais extraídas pelas camadas anteriores, juntam as peças do quebra-cabeça e calculam uma pontuação (*logits*) para cada uma das 10 classes do CIFAR-10, definindo a resposta final.

2.3 Métricas de Avaliação de Desempenho

Para avaliar matematicamente a qualidade do nosso classificador, utilizamos um conjunto de métricas estatísticas consolidadas:

- **Acurácia (*Accuracy*):** É a métrica mais intuitiva, representando a taxa geral de acertos. É calculada dividindo o total de imagens classificadas corretamente pelo número total de imagens avaliadas:

$$\text{Acurácia} = \frac{\text{Acertos Totais}}{\text{Total de Amostras}} \quad (1)$$

Apesar de simples, a acurácia isolada pode ser enganosa se o banco de dados for desequilibrado (por exemplo, se houvesse muito mais carros do que aviões). Como nosso *dataset* foi balanceado pelo processo de divisão estratificada, ela se torna altamente confiável.

- **Precisão (*Precision*):** Responde à pergunta: "De todas as imagens que o modelo disse que eram de uma determinada classe, quantas eram realmente daquela classe?".

É o indicativo de quão confiável o modelo é ao fazer uma afirmação, evitando falsos alarmes (falsos positivos).

$$\text{Precisão} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Positivos}} \quad (2)$$

- **Sensibilidade ou Revocação (*Recall*):** Responde à pergunta: "De todas as imagens que **realmente pertenciam** a uma classe na vida real, quantas o modelo foi capaz de encontrar e capturar?". Mede a eficiência em não deixar nenhuma imagem passar batida (falsos negativos).

$$\text{Sensibilidade} = \frac{\text{Verdadeiros Positivos}}{\text{Verdadeiros Positivos} + \text{Falsos Negativos}} \quad (3)$$

- **F1-Score:** Como muitas vezes a Precisão e a Sensibilidade competem entre si (melhorar uma pode piorar a outra), o F1-Score funciona como uma média harmônica balanceada entre as duas. É a métrica ideal para dar uma nota única e justa ao desempenho de cada classe.

$$\text{F1-Score} = 2 \times \frac{\text{Precisão} \times \text{Sensibilidade}}{\text{Precisão} + \text{Sensibilidade}} \quad (4)$$

- **Matriz de Confusão:** É uma tabela visual cruzada altamente informativa. Suas linhas representam as classes reais das imagens e suas colunas representam as previsões feitas pelo computador. Os acertos ficam concentrados na linha diagonal principal da matriz. Qualquer número fora dessa diagonal revela exatamente onde a rede está se "confundindo" (por exemplo, mostrando quantas vezes ela achou que um gato era, na verdade, um cachorro).

3 METODOLOGIA E VALIDAÇÃO

3.1 Papel da Validação no Desempenho do Classificador

A estratégia de validação é o pilar mais importante para garantir a qualidade de um sistema de Inteligência Artificial. Em nossa implementação, dividimos o banco de dados original através do método *Holdout Estratificado* nas seguintes proporções exatas: 70% para o conjunto de Treino, 15% para o conjunto de Validação e 15% para o conjunto de Teste. O termo "estratificado" significa que garantimos matematicamente que todas as 10 classes possuam a mesmíssima quantidade proporcional de fotos em cada um desses três pedaços, evitando favorecer qualquer categoria.

Para entender o papel de cada partição, podemos recorrer novamente à nossa analogia escolar:

1. **O Conjunto de Treino (70%):** São os livros didáticos e exercícios de fixação com gabarito que o aluno estuda diariamente. É aqui que o modelo ajusta seus parâmetros internos básicos.
2. **O Conjunto de Validação (15%):** Funciona como aqueles simulados periódicos realizados em sala de aula antes do exame oficial. O aluno não pode decorar o simulado (já que as questões mudam a cada rodada), mas o professor usa as notas do simulado para avaliar se o aluno está aprendendo de verdade ou apenas decorando. Se a nota do simulado começar a cair enquanto a nota dos exercícios de fixação continua 100%, o professor sabe que o aluno começou a sofrer de *overfitting*.
3. **O Conjunto de Teste (15%):** É o vestibular oficial de final de ano. Ele fica guardado em um cofre trancado e a máquina só tem acesso a ele uma única vez, no final de tudo. Serve para dar o veredito final e realista sobre a inteligência do sistema.

Em nosso pipeline de execução, o papel prático da validação se deu em três frentes críticas:

- **Seleção de Modelos (*Model Selection*):** Permitiu monitorar a evolução do aprendizado época por época.
- **Agendamento de Taxa de Aprendizado (*Learning Rate Scheduler*):** Usando o agendador *ReduceLROnPlateau* combinado com o otimizador Adam, a validação indicou o momento exato em que a rede precisava dar passos mais curtos e cuidadosos para refinar os detalhes finos do aprendizado, reduzindo automaticamente o lr ao detectar estagnação da acurácia de validação.
- **Parada Precoce (*Early Stopping*):** Configurada com uma paciência de 10 épocas, essa rotina atuou como um fiscal automatizado. Se a acurácia do conjunto

de validação ficasse 10 rodadas consecutivas sem apresentar melhorias, o treinamento era interrompido imediatamente e o melhor estado anterior da rede era salvo no arquivo `best_model.pth`, evitando que o modelo continuasse rodando e estragasse o aprendizado por excesso de decoreba (*overfitting*).

4 ANÁLISE DOS RESULTADOS

4.1 Comparação de Otimizadores (SGD e Adam)

Os dois experimentos utilizaram a mesma arquitetura, os mesmos dados e o mesmo agendador *ReduceLROnPlateau* (fator 0,5, paciência de 5 épocas sobre a acurácia de validação). A diferença entre eles foi exclusivamente o otimizador.

O primeiro experimento utilizou o SGD (*Stochastic Gradient Descent*) com taxa de aprendizado inicial de 0,01, momentum de 0,9 e decaimento de pesos de 5×10^{-4} . O treinamento convergiu de forma estável ao longo de 114 épocas, com o *scheduler* reduzindo o lr automaticamente em múltiplas etapas à medida que a validação estagnava: de 0,010 para 0,005 na época 34, depois para 0,0025 na época 40, para 0,00125 na época 64 e continuando a se reduzir progressivamente até aproximadamente 0,000078 nas épocas finais. O melhor resultado foi alcançado na época 104, com acurácia de validação de 87,99% e acurácia no conjunto de teste de 87,57%.

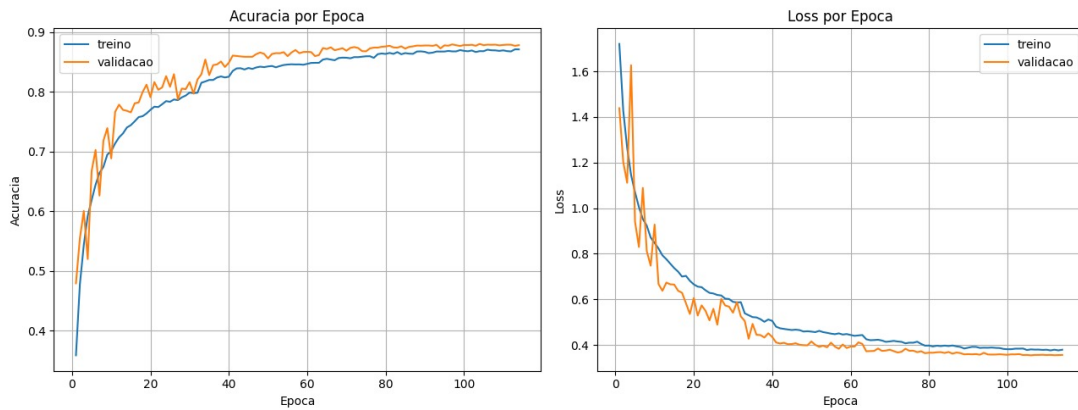


Figura 1: Curvas de acurácia e loss ao longo das épocas com o otimizador SGD.

O segundo experimento utilizou o Adam (*Adaptive Moment Estimation*) com taxa de aprendizado inicial de 1×10^{-3} e o mesmo agendador. O treinamento também convergiu de forma suave, com o lr passando de 0,001 para 0,0005 na época 48, para 0,00025 na época 70, para 0,000125 na época 85 e chegando a 0,000063 na época 95. O melhor resultado foi alcançado na época 95, com acurácia de validação de 86,90% e acurácia no conjunto de teste de 86,59%. O *early stopping* encerrou o treinamento na época 105.

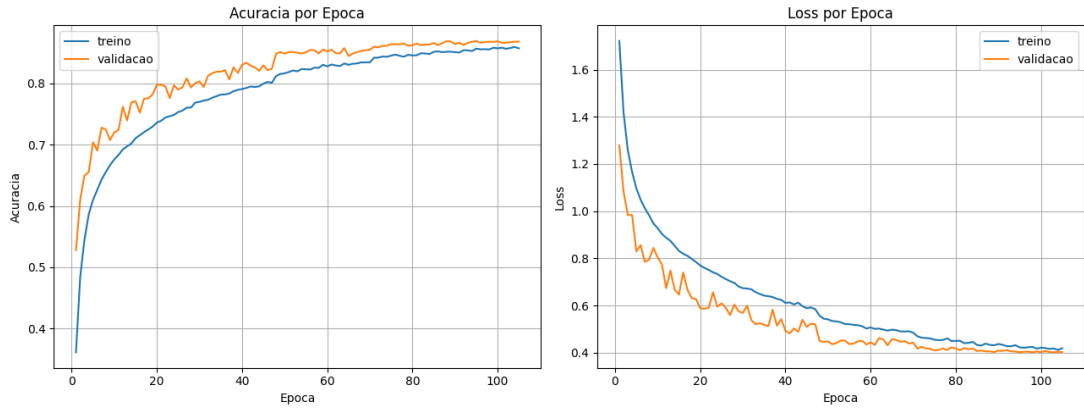


Figura 2: Curvas de acurácia e loss ao longo das épocas com o otimizador Adam.

Os dois otimizadores produziram treinamentos estáveis e resultados próximos, com o SGD saindo ligeiramente à frente em ambas as métricas. A diferença de aproximadamente 1 ponto percentual no teste (87,57% contra 86,59%) sugere que, com o agendador adaptativo controlando as reduções de lr, o SGD conseguiu extrair um refinamento um pouco maior do que o Adam ao longo das épocas finais.

4.2 Discussão do Impacto da Validação e do Treinamento

Durante todo o processo de treinamento, o conjunto de validação exerceu três funções simultâneas. A primeira foi a seleção do melhor estado da rede: a cada época, se a acurácia de validação superasse o melhor valor registrado até então, o estado dos pesos era salvo no arquivo `best_model.pth`. Isso garantiu que o modelo entregue ao teste final fosse sempre o de maior desempenho observado e não o da última época executada.

A segunda função foi alimentar o agendador de taxa de aprendizado. O *ReduceLROnPlateau* monitora continuamente a acurácia de validação e, ao detectar que ela estacionou, reduz o lr pela metade. É uma lógica parecida com a de um estudante que percebe que estudar do mesmo jeito não está mais trazendo resultado e decide mudar a estratégia. O modelo, ao atingir um platô natural, passou a dar passos mais finos e conseguiu extrair ganhos adicionais que de outra forma não seriam possíveis.

A terceira função foi controlar o *early stopping*. Com paciência de 10 épocas, essa rotina atuou como um limite de segurança: se o modelo não conseguisse melhorar a validação por 10 rodadas consecutivas, o treinamento era encerrado, evitando que a rede continuasse ajustando os pesos sem ganho real de generalização.

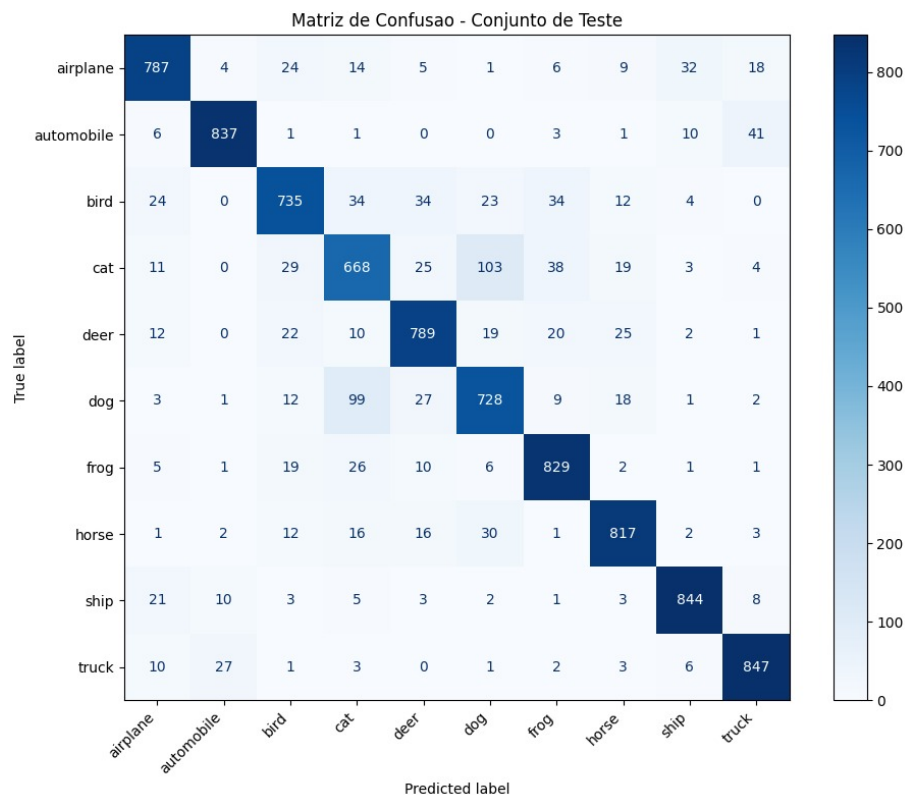


Figura 3: Matriz de confusão no conjunto de teste com o otimizador SGD.

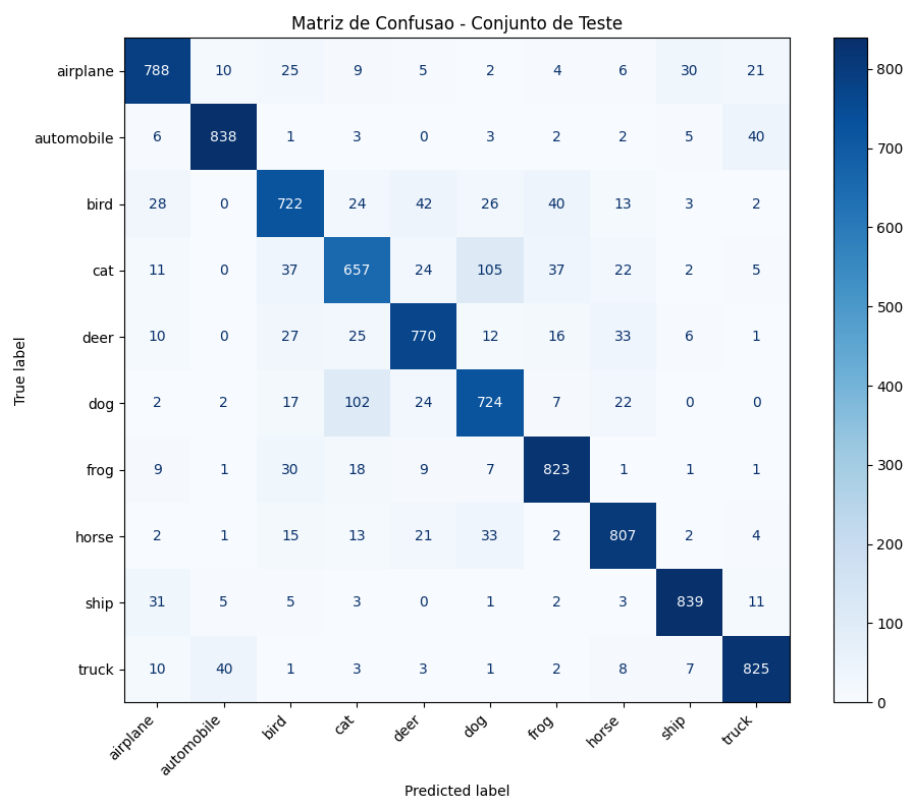


Figura 4: Matriz de confusão no conjunto de teste com o otimizador Adam.

O resultado prático dessa estratégia aparece nas curvas de treinamento. A acurácia de validação acompanhou de perto a de treino ao longo de todas as épocas, sem sinais evidentes de *overfitting*, o que indica que o modelo aprendeu os padrões reais das categorias e não apenas decorou os exemplos do conjunto de treino.

5 CONCLUSÃO

Este trabalho mostrou que construir um classificador de imagens eficiente vai muito além de definir uma arquitetura de rede. O pipeline completo, da divisão estratificada dos dados até o monitoramento contínuo pela validação, foi determinante para o resultado final obtido.

A comparação entre SGD e Adam mostrou que, quando ambos utilizam o mesmo agendador adaptativo, os dois otimizadores produzem treinamentos igualmente estáveis. O SGD, com lr inicial de 0,01 e decaimento de pesos, conseguiu um refinamento ligeiramente maior nas épocas finais e encerrou com resultado superior ao Adam nas duas métricas analisadas.

O melhor resultado geral foi obtido pelo SGD, com acurácia de validação de 87,99% e acurácia no conjunto de teste independente de 87,57%. O Adam atingiu 86,90% de validação e 86,59% no teste. Ambos os números são expressivos para uma CNN de aproximadamente 1,1 milhão de parâmetros aplicada ao CIFAR-10 e confirmam a hipótese central do projeto: a validação rigorosa, aliada ao agendamento adaptativo do lr e ao *early stopping*, é o principal mecanismo para garantir que o modelo aprenda a generalizar e não apenas a memorizar os dados de treino.

Como limitação, a arquitetura utilizada tem um teto natural de desempenho. Redes mais profundas como ResNet conseguem superar 90% no CIFAR-10, mas fogem ao escopo de uma CNN pequena proposto pelo enunciado.

REFERÊNCIAS

ULTRALYTICS. **CIFAR-10 Image Classification Dataset**. Ultralytics Docs, [s.d.]. Disponível em: <https://docs.ultralytics.com/pt/datasets/classify/cifar10/>. Acesso em: 26 maio 2026.

LEMES, Dimas Augusto Mendes. **Aula 03 - Aprendizado de Máquina: Inteligência Artificial e Aprendizado de Máquina**. Material de aula, 2026. Disponível em: https://puc-campinas.instructure.com/courses/91428/files/3887265?module_item_id=557013. Acesso em: 26 maio 2026.

LEMES, Dimas Augusto Mendes. **Aula 04 - Aprendizado Profundo: Inteligência Artificial e Aprendizado de Máquina**. Material de aula, 2026. Disponível em: <https://puc-campinas.instructure.com/courses/91428/files?preview=4061118>. Acesso em: 26 maio 2026.

CS231n: Deep Learning for Computer Vision. **Convolutional Neural Networks for Visual Recognition**. Stanford University, [s.d.]. Disponível em: <https://cs231n.github.io/convolutional-networks/>. Acesso em: 27 maio 2026.

PYTORCH. **Training a Classifier**. PyTorch Tutorials, 2017. Disponível em: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html. Acesso em: 27 maio 2026.

PYTORCH. **Transfer Learning for Computer Vision Tutorial**. PyTorch Tutorials, 2017. Disponível em: https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html. Acesso em: 27 maio 2026.

PYTORCH. **Transforming and augmenting images**. Torchvision v0.17 Documentation, [s.d.]. Disponível em: <https://pytorch.org/vision/stable/transforms.html>. Acesso em: 27 maio 2026.

PYTORCH. **Torch.utils.data**. PyTorch Documentation, 2025. Disponível em: <https://pytorch.org/docs/stable/data.html>. Acesso em: 27 maio 2026.

PYTORCH. **Torch.optim**. PyTorch Documentation, 2025. Disponível em: <https://pytorch.org/docs/stable/optim.html>. Acesso em: 27 maio 2026.

PYTORCH. **torch.optim.SGD**. PyTorch Documentation, [s.d.]. Disponível em: <https://docs.pytorch.org/docs/2.12/generated/torch.optim.SGD.html>. Acesso em: 27 maio 2026.

PYTORCH. **torch.optim.lr_scheduler.MultiStepLR**. PyTorch Documentation, [s.d.]. Disponível em: https://docs.pytorch.org/docs/2.12/generated/torch.optim.lr_scheduler.MultiStepLR.html. Acesso em: 27 maio 2026.

SCIKIT-LEARN. **sklearn.model_selection.train_test_split**. scikit-learn v1.4 Documentation, [s.d.]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. Acesso em: 27 maio 2026.

SCIKIT-LEARN. **sklearn.metrics.confusion_matrix**. scikit-learn v1.4 Documentation, [s.d.]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html. Acesso em: 27 maio 2026.

SCIKIT-LEARN. **sklearn.metrics.classification_report**. scikit-learn v1.4 Documentation, [s.d.]. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html. Acesso em: 27 maio 2026.